Interaktivt webbprojekt - Translators AB

1. Säkerhetsanalys (hot och risker)

XSS (Cross-Site Scripting)

1. DOM-baserad XSS
2. Lagrad XSS

Varje identifierad risk kräver en kombination av tekniska skyddsåtgärder, både i frontend och backend.

Översikt över skyddsåtgärder:

DOM XSS - textContent, CSP

Lagrad XSS - server-side encoding

Lösenord - bcrypt,

Brute force - rate limiting,

Åtkomst - serverkontroller,

Session - säkra cookies.

Risk för DOM-baserade XSS attacker

Det oskyddade inmatningsfältet `#inputMeddelande` är ett fält öppen för inmatning av skadlig kod och innebär risk för DOM-baserad XSS.
DOM-baserad XSS kan uppstå. Ingen server är inblandad, datan skulle kunna komma via ingripandens inmatning i input-fältet, JS i användarens webbläsare skulle kunna stoppa in en skadlig kod in i sidan via `.innerHTML` och webbläsaren tolka det som kod och manipulera DOM.

Problem fanns i koden:

```
document.getElementById("output").innerHTML = meddelande;
```

En DOM-baserad skickas aldrig till servern och är svåra att upptäcka av brandväggar för webbaserade program (WAF) och syns inte i serverloggar.[1]

Åtgärder

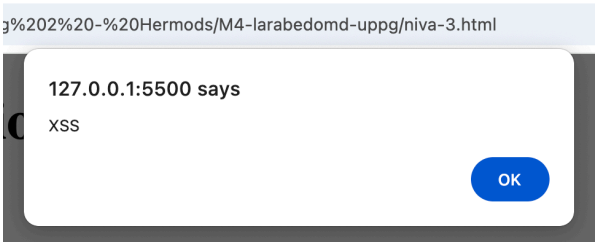
1. Först testade jag vanlig användar-/text-input i inmatningsfältet för att testa funktionen.
2. Därefter matade jag in:

a) `<script>alert('Du har blivit hackad!')</script>`

Chrome visar inte någon alert popp up men input via `.innerHTML.` Input tolkas här som HTML-kod och webbläsaren bygger DOM-element av det. Detta syns i Dev Tools:

```
<button onclick="visaMeddelande()">Visa</button>
...  ▼<div id="output"> == $0
       <script>alert('Du har blivit hackad!')</script>
```

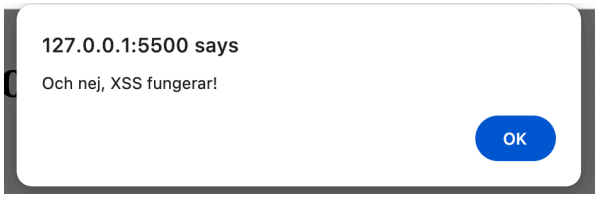
Skript i `<script>`-taggar körs normalt inte automatiskt när de sätts in via innerHTML i moderna webbläsare. Däremot kan andra typer av HTML-innehåll, till exempel element med händelseattribut som `onerror,` fortfarande leda till XSS.



[1] Vad är en XSS-attack? Definition och förklaring. https://www.kaspersky.se/resource-center/definitions/what-is-a-cross-site-scripting-attack (hämtad 2026-03-14)

Därför testade jag också: `<IMG SRC=/ onerror="alert(String.fromCharCode(88,83,83))"></img>`[2]

och `<img src=x onerror="alert('Och nej, XSS fungerar!')">`



`<img onerror>- testet fungerar som XSS.`

Säkerhetsåtgärd, i JS.

I stället för `innerHTML` lägger jag in .textContent i JS, för då tolkas inte input som HTML utan som ren text.

```
document.getElementById("output").textContent = meddelande;
```

Det skapas inget DOM-element som kan trigga `onerror.`

När jag bytte från innerHTML till textContent i JS visades användarens inmatning fortfarande på sidan som vanlig text, men utan att tolkas som HTML. Det minskade risken för DOM-baserad XSS. När jag därefter lade till en Content Security Policy slutade knappen först att fungera, eftersom min kod använde inline-JavaScript i både onclick och ett `<script>`-block. CSP blockerade alltså denna typ av skriptkörning. För att få både funktion och säkerhet behöver JavaScript därför flyttas till en extern fil och händelsen kopplas med `addEventListener.` Detta ger ett bättre skydd i flera lager.

Säkerhetsåtgärd 2

Ett extra skyddslager är **Content Security Policy (CSP)** I <head>:

```
<meta http-equiv="Content-Security-Policy" content="default-src
```

[2] OWASP. XSS Filter Evasion Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/XSS_Filter_Evasion_Cheat_Sheet.html (hämtad 2026-03-14)

```
'self'; script-src 'self'; img-src ‘self'; style-src 'self';">
```

CSP skyddar mot säkerhetshot som Cross-Site Scripting (där angripare försöker initierar skadlig kod), data-stöld (genom att begränsa vilka externa tjänster kan nås), malware-distribution (genom att hindra från att farlig kod laddas in).

CSP-regeln kontrollerar vilka resurser såsom skript, stilar m.m. webbplatsen får ladda genom att begränsa vilka resurser webbsidan får ladda. Resurser som laddas får bara ha samma ursprung/domän, medan resurser från externa servar är inte tillåtna som standard.
JS får bara köras från samma webbplats, extern JS blockeras. Bilder får bara laddas från samma webbplats, medan bilder från andra domäner blockeras. Stilar (CSS) får endast ha samma ursprung som webbsidan, och eftersom `styles.css` ligger i samma domän som resten av webbplatsen kommer det att godkännas av CSP:n. Font från extern källa behöver laddas ned eftersom det annars kommer att blockeras av CSP.
Eventuella inline CSS-attribut blockeras också, oavsett om dessa placeras i <body> eller i <head>, och även om det finns förfaringssätt för att behålla CSS i html-dokumentet så rekommenderar jag att all CSS läggs i en egen fil.

Ytterligare skydd via CSP kan vara:

- `Object-src ‘none’` - blockerar alla `<object>`, `<embed>` och `<applet>`. Inga pluginbaserade objekt får laddas alls. Tidigare var det Flash, Java applets och pdf-plugins.
- `Base-uri ‘self’` - begränsar användningen av `<base>` så att en angripare inte kan ändra hur relativa länkar och formuläradresser tolkas.
- `Form-action ‘self”` – ser till att formulär bara skickas till egen server.

  En alltför strikt policy med `script-src'self'` blockerar externa skript helt, inklusive analysprogram såsom Google Analytics. För att ändå kunna ladda Analytics efter användarens samtycke behöver man uttryckligen tillåta scriptkälla i `script-src.` På så sätt blockeras fortfarande okända externa skript, medan den specifika och betrodda källan för analysverktyget tillåts.

  I CSP-koden föreslås följande undantag:

- i `Script-src –` tillåter Analytics-skriptet att laddas från extern tjänst.

- i `Connect-src` - tillåter att sidan skickar analysdata till analystjänsten.
- i `img-src` - analysprogram kan ibland använda bild/piexalanrop.

Säkerhetsåtgärd 3

Som ytterligare skydd kan inmatningen valideras eller filtreras innan den visas, exempelvis genom att ersätta tecken som **`"<"`** och **">"** i JS-koden så att de inte kan tolkas som kod. Användarens input visas redan som text, så **sanitize**-funktionen är mer ett extra skyddslager.

Säkerheten bygger alltså på flera lager: säker DOM-hantering, begränsning av resurser via CSP, men även i försiktig hantering av användarinmatning.

Validering

Validering är till för att säkerställa att bara säker, korrekt data behandlas. Det finns klientvalidering (i webbläsaren) och servervalidering (på servern).

HTML5 har en inbyggd validering. Webbläsaren validerar epost automatiskt vid `type="email",` och pattern="[a-zåäöA-ZÅÄÖ\s'-]+" tillåter bokstäver, mellanslag, bindestreck och apostrofer. HTML-validering är dock bara ett filter, inte säkerhet eftersom det kan kringgås. En angripare kan exempelvis öppna utvecklarverktygen och ta bort required och pantern eller skicka data direkt till servern utan att gå igenom HTML-formuläret. JS-validering ger ett andra försvar och fångar både misstag och försök att kringgå säkerheten. HTML- och JavaScript-validering förbättrar användarupplevelsen och fångar många fel tidigt, men utgör inte ett tillräckligt säkerhetsskydd. Den verkliga säkerhetskontrollen måste ske på serversidan.

Risk för lagrad XSS

En kundportal med inloggning och användarspecifika dokument innebär flera säkerhetsrisker.
En risk är broken access control (IDOR), där en användare kan försöka komma åt någon annans data genom att ändra id i en länk, till exempelvis /dokument/1234. Därför måste servern vid varje begäran kontrollera att användaren har rätt att se just det dokumentet.
Inloggningsfunktionen är sårbar för brute force-attacker, där angripare testar många lösenord eller återanvänder lösenord från tidigare dataläckor. Skydda detta genom:
Hash (t.ex. med bcrypt) och salt på lösenord (vilket gör att de inte går att läsa även om databasen läcker), att begränsa antalet inloggningsförsök, att logga misstänkt aktivitet och rekommenderad tvåfaktorsautentisering.
Vid behov kan man använda **CAPTCHA.** Det är en säkerhetskontroll som används för att skilja människor från automatiska program (botar), för att stoppa brune force-attacker och automatiserade inloggningsförsök.
Det kan vara bra att skydda sessioner med säkra cookie-inställningar. Detta uppnås genom inställningarna HttpOnly (hindrar JavaScript från att läsa cookien), Secure (skickas endast över HTTPS) och SameSite (minskar risken att cookies skickas med förfrågningar från andra webbplatser).

Om portalen hanterar sekretessbelagda dokument behövs extra skydd både vid överföring och lagring. Använd HTTPS för all datakommunikation och skydda filer på serversidan med åtkomstkontroll, loggning och eventuell kryptering. Frontend-skydd som CSP och inputvalidering är viktiga men räcker inte ensamma.

2. Tillgänglighetsanpassning

Vi byggande av webbsidan behöver utvecklare tänka på de av W3C (World Wide Web Consortium) fastställda, fyra principer (Webb Content Accessibility Guidelines). All information och användargränssnitt på sidan behöver kunna uppfattas (“perceivable”), användare måste kunna hantera alla användargränssnitt och navigering (“operable”). Vidare behöver användare kunna förstå

all information och alla användargränssnitt på sidan (“understandable”, till exempel om det inträffar ett fel när användaren klickar på “Submit”-button och användaren får ett felmeddelande behöver hen kunna förstå vad felet betyder och vad man ska göra för att komma vidare med det.[3]

Navigering via <tab>

Man knappar oftast mellan länkar, knappar, formulärfält och andra interaktiva element. På den här sidan går det att tabba mellan URL-fältet, inputfältet och knappen. Sidan bör använda semantiskt HTML, såsom <button>, <section> med flera, och sidan har flera <html> element som förslag, vilka är att föredra framför icke-semanstisk HTML element (såsom t.ex.).

Cookie-bannern har gjorts mer tillgänglig genom att använda en dold rubrik. Rubriken är visuellt dold med en så kallad visually hidden-teknik men finns kvar i dokumentstrukturen för skärmläsare. På så sätt kan användare som navigerar via rubriker identifiera sektionen.
Sektionen kopplas till rubriken genom attributet `aria-labelledby.` Ett alternativ hade varit att använda `aria-label,` men metoden med rubrik ger en bättre semantisk struktur eftersom rubriker kan användas för navigering i skärmläsare. Tekniken att visuellt dölja text men behålla den för assistentiv teknik är i överensstämmelse med W3C. [4]
För `<div id=“output">` använder jag attributet `aria-live="polite"` vilket gör att skärmläsare automatiskt läser upp förändringar i innehållet när JS uppdaterar elementet. Detta är viktigt vid dynamiskt innehåll eftersom användare annars inte får någon information om att något har ändrats. På exempelsidan Translators AB, placeras output direkt på sidan, för att visa innehållet för inmatningen.

Den fjärde WCAG principen, robusthet, ställer krav på att sidans innehåll ska vara tillgänglig med hjälpmedel och andra användaragenter och att arbete med tillgänglighet behöver vara dynamisk och webbsidans teknik hänga med när dessa tekniker utvecklas.

I dagsläget, år 2026 använder användare sig av exempelvis skärmläsare NVDA (vid Windows).[5]

---

[3] Belfrage Paul, Webbutveckling 1, 2025, s. 196

[4] Rekommenderad WCAG 2.1 teknik: https://www.w3.org/WAI/WCAG21/Techniques/css/C7

[5] NVDA skärmläsare för Windows, https://www.nvaccess.org/

Personer som använder skärmläsare kan ofta hoppa mellan landmarks, regioner, rubriker. När en

sektion är märkt med `aria-label` så blir navigationen tydligare.

(Knappen med id="button-m" markerade jag först med `aria-label` för skärmläsare, vartefter jag tog bort markeringen eftersom jag bedömde att knappen redan hade ett namn genom sin synliga text "Skicka", vilket är tillräckligt uttrycksfullt för en skärmläsare.)

Vanligtvis får icke-semantiska html-element såsom ingen fokus. `Tabindex="0"` bör användas när ett ickelement behöver förtydligas ytterligare.

Hantering av fokus för Cookie-bannern

Om webbplatsen använder inloggning behövs ofta en sessionscookie för att hålla användaren inloggad. En sådan cookie kan vara tekniskt nödvändig och kräver därför normalt inte samtycke på samma sätt som analyscookies. Däremot måste den skyddas med säkra inställningar, till exempel HttpOnly, s och lämplig SameSite, så att risken för sessionkapning minskar.

För att säkerställa att användare som navigerar med tangentbord eller skärmläsare inte missar cookie-bannern märks sektionen märkts med tabindex="-1". Normalt kan en <section> inte ta emot fokus, men med detta attribut kan fokus flyttas programmatiskt med JS (element.focus()) när sidan laddas.

När fokus flyttas till bannern via JS börjar skärmläsaren läsa upp innehållet i sektionen. I detta fall läses först rubriken upp, följt av texten och knapparna, exempelvis:

- "Cookies."
- "Vi behöver ditt samtycke för att använda analys-cookies".
- "Acceptera cookies".
- "Neka cookies."

Genom att kombinera detta med attributet `aria-labelledby` läser skärmläsaren automatiskt upp bannerns rubrik när fokus flyttas till sektionen. På så sätt uppmärksammas användaren direkt på att ett val om cookies behöver göras.

Eftersom värdet -1 används hamnar inte bannern i den vanliga tabb-ordningen. Efter att användaren har gjort sitt val kan hen fortsätta navigera vidare på sidan utan att oavsiktligt tabba tillbaka till bannern. Cookie bannern försvinner efter valet, och minnet om valet kommer finnas i LocalStorage inför nästa besök.

Färgkontrast

När man har `rgba(…)` ovanpå bakgrund så blandas färgen. I ett vanligt projekt hade man behövt vara ännu mera noggrann med exakta beräkningar av lager på lager. Men i de flesta fall på denna sida ligger ovarlays på ljus bakgrund och jag tog fram hex-färger inför tester som ligger nära nyansmässigt.

Samtliga färgkombinationer nedan är godkända för normal och stor text samt grafiska objekt enligt WCAG AA och WCAG AAA.[6]

1. Text vs kort

Text: #1F1B16
Bakgrund: #FFFDF6

Contrast Ratio: 16.82:1

2. H1
Text: #1F1B16
Bakgrund: #F7F7F7

Contrast Ratio: 15.98:1

3. Navigation
Text: #1F1B16
Hover: #F0E9DF

Contrast Ratio: 14.2:1

4. Knappar
Text: #1F1B16
Bakgrund: #EFE2C8

Contrast Ratio: 13.35:1

[6] WebAim Contrast Checker, https://webaim.org/resources/contrastchecker/

5. Footer
Text: #1F1B16
Bakgrund: #F6F6F6

Contrast Ratio: 15.84:1

6. Form inputs
Text: #1F1B16
Bakgrund: #FFFFFF

Contrast Ratio: 17.12:1

Radavstånd mellan linjer bör vara minst 1.5, satt i <body> i CSS, i enlighet med principen på att innehåll ska vara upptäckbar/synlig (perceivable).

3. Hållbarhet

Energikrävande animation har tagits bort. Den anses dessutom inte fylla någon annan en eventuell en estetisk funktion, men tar för mycket av uppmärksamhet från innehållet i sidan och är inte ändamålsenligt. Vill man ha animationen kvar bör bakgrunder följa organisationens grafiska profil. Om bakgrundskortet (.card) tas bort eller dess bakgrund ändras behöver nya färgkontrast tester göras mellan text och bakgrund för att uppfylla krav på tillgänglighet.

Optimering av bilder

Bilder står ofta för den största datamängden på en webbsida. Jag laddade ned en bild med skrivmaskin på ett skrivbord i mediumformat (800 x 450) på 81KB och omvandlade denna via sqoosh.app till WebP för att minska filstorleken. Med Quality minskad till 75% blev och nytt filformat blev besparingen på 59% till 35KB. Bilden blev också sparad som MozJPEG och också vikt på 35KB, och för ändamålet godkänt kvalitet. Till skillnad från format som WebP eller AVIF, som kräver att webbläsaren har stöd för just det specifika formatet, är MozJPEG en extremt effektivt packad JPEG.

Bakgrundsbilden optimerades via sqoosh.app och kunde minskas utan för stor kvalitetsbortfall från 1.2MB i ursprunglig .jpg-format till 200KB som WebP. I .jpg kunde väger bilder 197kB, men med betydligt större kvalitetsbortfall än om det omvandlas till WebP, något som kan vara estetiskt mer eller mindre acceptabelt när en ljus eller mörk linjär-gradient läggs på bakgrundsbilden (se förslag i

styles.css).

Det etiska förhållningssättet är att få alla att ta del av innehållet på sidan, och även de som använder äldre webbläsare. Modern CSS tillåter även webbläsare som inte kan ladda .webp att läsa .jpg i stället, och optimerad bakgrundsbild i .jpg finns med i image-mappen.

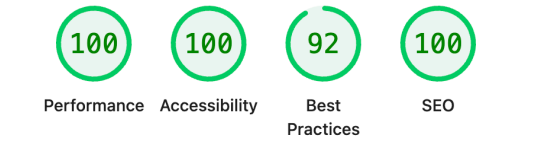
Jag använde även attributet loading="lazy" så att bilden bara laddas när den behövs. loading="lazy" kan minska både dataöverföring, CPU-användning och energiförbrukning eftersom bilder längre ned på sidan inte behöver laddas förrän de faktiskt behövs.

Kodoptimering

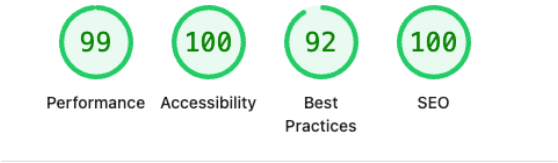
En annan åtgärd för att minska energiförbrukningen är att minifera mängden data som skickas över nätverket. Detta kan göras genom minifiering av HTML, CSS och JavaScript. Vid minifiering tas onödiga mellanslag, radbrytningar och kommentarer bort från koden, vilket gör filerna mindre. Mindre filer innebär att mindre data behöver överföras mellan server och användare, vilket minskar både laddningstid och energiförbrukning.

Ett annat förslag för att minska miljöpåverkan är att sidan läggs på en server som drivs av göra energi (grön hosting).

Resultat av tester:



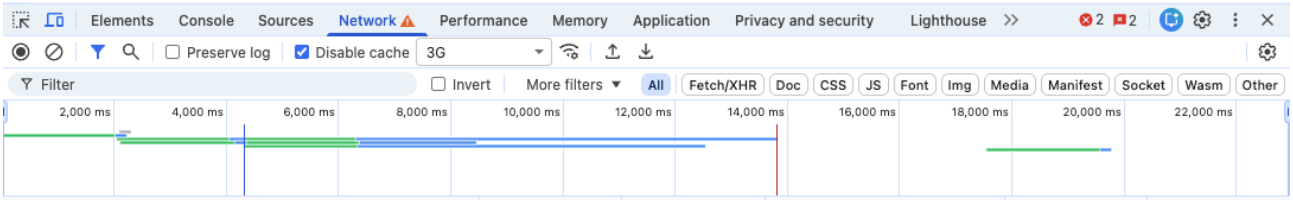
För desktopversionen (Lighthouse)



För den mobila versionen (Lighthouse)

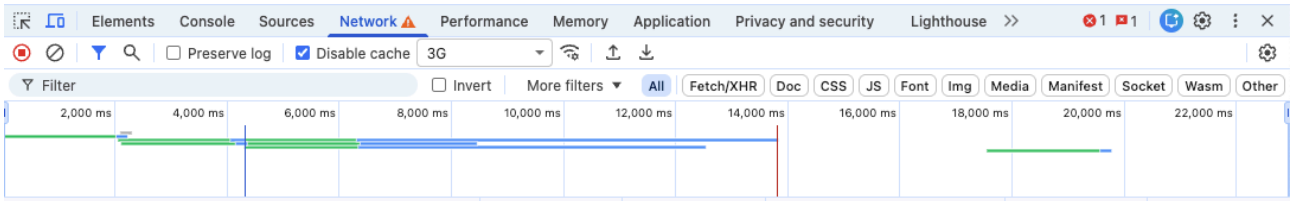
Prestandan har testats testats i Chrome DevTools och Lighthouse. I Network-panelen användes strypt nätverkshastighet (3G) för att simulera långsammare uppkoppling. Mätningen gjordes lokalt och ger därför inte en helt exakt bild av hur sidan skulle laddas från en publicerad server, men den ger ändå ett användbart underlag för att bedöma resursstorlek, laddningsordning och möjliga förbättringar.

Test för 1046 x 63 (Responsive)



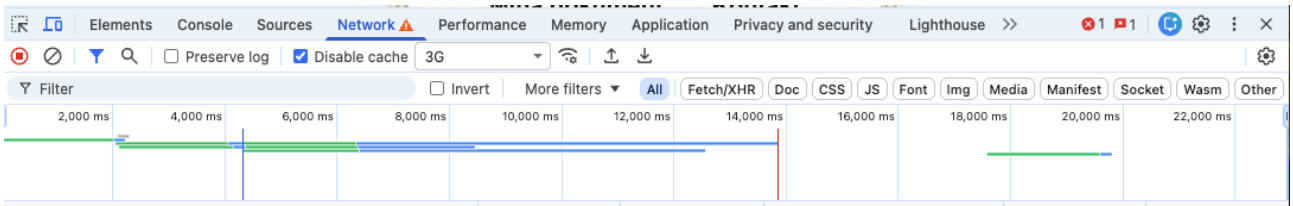
Name	Status	Type	Initiator	Size	Time
niva-3.html	200	document	Other	6.9 kB	2.14 s
styles.css	200	stylesheet	niva-3.html:8	9.4 kB	2.25 s
axe.js	200	script	content.bundle.js:2	561 kB	1 ms
highlighter.js	200	script	content.bundle.js:61	21.6 kB	1 ms
script.js	200	script	niva-3.html:117	2.2 kB	2.10 s
js?id=G-XXXX	200	script	script.js:32	137 kB	8.22 s
buch.jpg	200	jpeg	styles.css	202 kB	9.51 s
skrivmaskin.webp	200	webp	niva-3.html:117	35.2 kB	4.13 s
collect?v=2&tid=G-XXXX&gtm=45je63g0h1za200&_p=1773...dt...	204	fetch	js?id=G-XXXX:134	0.0 kB	2.03 s

Test för 820 x 1180 (iPad Air)



Name	Status	Type	Initiator	Size	Time
niva-3.html	200	document	Other	6.9 kB	2.14 s
styles.css	200	stylesheet	niva-3.html:8	9.4 kB	2.25 s
axe.js	200	script	content.bundle.js:2	561 kB	1 ms
highlighter.js	200	script	content.bundle.js:61	21.6 kB	1 ms
script.js	200	script	niva-3.html:117	2.2 kB	2.10 s
js?id=G-XXXX	200	script	script.js:32	137 kB	8.22 s
buch.jpg	200	jpeg	styles.css	202 kB	9.51 s
skrivmaskin.webp	200	webp	niva-3.html:117	35.2 kB	4.13 s
collect?v=2&tid=G-XXXX&gtm=45je63g0h1za200&_p=1773...dt...	204	fetch	js?id=G-XXXX:134	0.0 kB	2.03 s

Test för 430 x 932 (iPad Pro Max)



Name	Status	Type	Initiator	Size	Time
niva-3.html	200	document	Other	6.9 kB	2.14 s
styles.css	200	stylesheet	niva-3.html:8	9.4 kB	2.25 s
axe.js	200	script	content.bundle.js:2	561 kB	1 ms
highlighter.js	200	script	content.bundle.js:61	21.6 kB	1 ms
script.js	200	script	niva-3.html:117	2.2 kB	2.10 s
js?id=G-XXXX	200	script	script.js:32	137 kB	8.25 s
buch.jpg	200	jpeg	styles.css	202 kB	9.54 s
skrivmaskin.webp	200	webp	niva-3.html:117	35.2 kB	4.14 s
collect?v=2&tid=G-XXXX&gtm=45je63g0za200&_p=177377...dt...	204	fetch	js?id=G-XXXX:134	0.0 kB	2.02 s

Enligt Networkscheck har bakgrundsbilden buch.jpg en nedladdningstid på 9.54s, vid simulering av användning med 3G nätverk på iPhone 14 Max och på 9.51s för iPadAir, vilket är för mycket.

I CSS för bakgrundsbilden ges alternativ för webbläsare att välja mellan bildformat .jpg och WebP. Bakgrundsbilden buch.webp på 200KB och buch.jpg på 202KB kan vi behov eventuellt bytas ut mot en annan, eller mot en bild i .svg format. Fördelen med .svg är oändlig skalbarhet utan kvalitetsförlust. Vektorbilder brukar vara betydligt lättare än WebP. Enkla former, linjer och plana färger beskrivs som kod, vilket kan resultera i filstorlek under 1KB, men med mera komplexa illustrationer kan filstorleken snabbt växa och bli tyngre än en optimerad WebP. För fotografiska bilder är .svg dock inte lämplig som format.

Vidare bör CSS, HTML och JS minifieras, men för den här uppgiften lämnades dessa i sin orignalstorlek.

Ytterligare tester för flera enheter behöver göras.

4. Överensstämmelse med annan regelverk/lagstifning

Val av bilder

En personuppgift är allt som kan identifiera en specifik person inklusive kontext enligt General Data Protection Regulation (GDPR). Som exempel valde jag en bild utan personer, eftersom fotografier av identifierbara människor kan räknas som personuppgifter. Genom att använda en bild på ett objekt undviks potentiella integritetsproblem. Bilden bör helst passa till den information man vill förmedla, och det kan också vara av symbolisk art. Informationsbärande bilder bör ha en meningsfull alt-text. Rent dekorativa bilder kan istället ha tom alt="", så att de inte stör skärmläsare.

Bilden på sidan och bakgrundbilden faller under lagen (1960:729) om rätt till litterära och konstnärliga verk (URL). Kan bilderna klassas som konstverk och uppfyller så kallad verkshöjd blir de skyddade som verk. Fotografiet på skrivmaskinen, i den mån fotot är tagen med en kamera kan

eventuellt bli skyddad som så kallad närstående rättighet enligt fotorätten i 49 §URL, om fotografiet inte anses vara ett verk. I bägge fall har skaparen en ideell och ekonomisk rätt och ensamrätt att förfoga över sina bilder, och vari ingår att mångfaldiga/kopiera bilder, bearbeta dessa, visa dessa och sprida dessa offentligt i olika medier. Skyddstiden för upphovsrätten gäller 70 år efter upphovsmannens död, och för närstående rättigheten i 50 år efter det att bilden togs.

När en webbläsare laddar ned en bild skapas en kopia av bilden på användarens enhet, och bilden görs samtidigt tillgänglig för webbplatsens besökare.
Licenser är upphovsmännens och fotografers frivilliga avtal om att i mer eller mindre utsträckning erbjuda andra att förfoga över deras bilder utan att betala för det. Det finns olika typer av licenser med såväl friheter som inskränkningar i förfogandet för bilder, filmer, ljud och andra alster. [7]
Man behöver alltid kontrollera om nedladdningsplatsen erbjuder giltiga licenser, och om dessa inte finns be skaparen om lov före användning.

Bilden på sidan och bakgrundsbilden faller under Creative Commons, CC0-licens och de får både kopieras och visas offentligt även i kommersiellt syfte.

Upphovsrätt och närstående rättigheter

Datainsamling via formulär

Ur ett etiskt perspektiv är det viktigt att skydda användarens integritet och inte samla in mer information än nödvändigt. Därför använder formuläret endast ett enkelt meddelandefält och bara de personuppgifter bör samlas som behövs för ändamålet, såsom en första kontakt.

Användning av webbanalys-verktyg, information och samtycke
Enligt ePrivacy-direktivet behöver webbplatsen informera om och inhämta samtycke redan innan ett trafik-analysprogram, t.ex. Google Analytics eller en europeisk motsvarighet såsom Matomo, Plausible Analytics eller Fathom Analytics används. Besökaren behöver aktivt godkänna cookies som inte är att anse nödvändiga, med andra ord ska besökaren kunna neka statistikcookies.
Google Analytics är inte förbjudet enligt GDPR, eftersom programmet nu har automatisk IP-

[7] Se exempel på licenser på https://creativecommons.org/share-your-work/cclicenses/

anonymisering, men som med vilket annat webbanalys-verktyg, man måste informera användaren vilka data som samlas in, varför, vem som får tillgång till dem, hur länge de sparas, och hur man kan säga nej.

Information och samtycke

Information till besökaren kan till exempel ske genom en cookie-banner och en integritetspolicy. Där anges vilka uppgifter som samlas in, exempelvis IP-adress, webbläsare, besökta sidor och besökstid. Det räcker dock inte med att informera besökaren. Analysverktyg får inte aktiveras förrän användaren har gett sitt samtycke. Med andra ord är det först efter det att användaren har godkänt analysverktyg som skriptet från exempelvis Google Tag Manager/gtag.js får köras.

Google Analytics implementeras vanligtvis med JavaScript-biblioteket gtag.js, vilket laddas från Googles servrar. Installationskoden tillhandahålls i Googles officiella utvecklardokumentation.[8] På webbsidan finns dock den enklase JavaScript kod versionen som inte tar hänsyn till att användaren bör samtycka att analysprogrammet körs (i det här projektet exempelvis via ett cookie-samtycke). Skriptet i <head> med hade kunnat aktiverats direkt när sidan laddas utan att användaren har accepterat datainsamlingen, om inte det hade stoppas av Content Security Policy, för då körs det inget analysprogram.

Se i stället förslag på “Alternativ skript” i script.js. Först när användaren klickar på Acceptera så laddas en analystjänst, t.ex. Googles bibliotek och gtag(‘config’) laddar analysprogrammet.

Ökad hänsyn till användaren

Cookie-bannern är utformad som en informationsbanner och inte som en så kallad modal dialog. Det innebär att användaren fortfarande kan navigera vidare på sidan efter att ha sett informationen. En modal dialog skulle istället blockera resten av sidan och kräva att användaren gör ett aktivt val innan fortsatt navigation är möjlig.
WCAG ställer krav på att användaren ska kunna förstå informationen, navigera med tangentbord och ta ett informerat beslut. GDPR (artikel 4.11 och 7) anger samtidigt att samtycke måste vara

[8] Länk till installationssida från Google Analytics. Set up the Google tag with gtag.js. https://developers.google.com/tag-platform/gtagjs

frivilligt, informerat, specifikt och otvetydigt. Det är till exempel inte tillåtet med förkryssade rutor, dolda val eller lösningar där det är svårare att neka än att acceptera cookies.

Det är däremot juridiskt möjligt att informera användaren och förvänta sig en aktiv handling, till exempel att acceptera eller neka cookies. I EU-praxis gäller dock oftast att användaren ska kunna neka analyscookies och ändå fortsätta använda webbplatsen. Om tillgången till sidan begränsas tills cookies accepteras anses samtycket i många fall inte vara frivilligt.

Det finns även exempel på så kallade dark patterns, där designen påverkar användaren att välja ett visst alternativ. Ett exempel är banners som blockerar sidan tills användaren gör ett aktivt val, samtidigt som det är tidskrävande att stänga av många förvalda cookies i inställningarna. Sådana lösningar kan dessutom uppfattas som oetiska och riskerar att försämra användarupplevelsen.

Ur användarperspektiv är det därför viktigt att gränssnittet inte upplevs som påträngande eller vilseledande. Även om lagstiftningen tillåter att en användare gör ett aktivt val för att fortsätta använda en webbplats kan det i många fall vara bättre för användarupplevelsen att inte blockera hela sidan med en banner.

Samtidigt bör det tydligt framgå att vissa cookies kan vara tekniskt nödvändiga för att webbplatsen ska fungera. Sådana cookies kräver normalt inte samtycke enligt EU:s cookie-regler.

Utöver detta måste GDPR i övrigt följas. Det innebär bland annat att det måste finnas en rättslig grund för datainsamling, att samtycke till behandling av personuppgifter ska vara frivilligt och aktivt samt att det ska finnas tydliga rutiner för att radera uppgifter om användaren begär det.

När det gäller kontaktformuläret kan insamling av vissa uppgifter vara nödvändig för att kunna svara på användarens meddelande. Informationen om syftet med denna datainsamling kan därför presenteras som ett informativt meddelande, exempelvis: “Vi kommer att använda dina uppgifter för att svara på ditt meddelande”, snarare än som ett samtycke som användaren måste godkänna.

Användaren bör också kunna använda formuläret utan att samtidigt ge sitt samtycke till marknadsföring. Checkboxen för marknadsföring:“Jag godkänner att mina uppgifter används för marknadsföring från Organisation AB”, den bör därför vara avmarkerad från början och inte utformas på ett vilseledande sätt. Attributet required bör inte användas i detta fall, eftersom

samtycket ska vara frivilligt. Annars skulle användaren i praktiken tvingas acceptera marknadsföring för att kunna skicka sitt meddelande via formuläret.

Kortfattad reflektion om säkerhet, etik och hållbarhet.

I arbetet med webbplatsen har jag försökt balansera säkerhet, tillgänglighet och hållbarhet. Ur ett etiskt perspektiv är det viktigt att skydda användarens integritet och inte samla in mer information än nödvändigt. Formuläret innehåller därför endast ett enkelt meddelandefält och lagrar ingen data. För att undvika integritetsproblem har jag valt bilder utan identifierbara personer, eftersom sådana bilder kan räknas som personuppgifter enligt GDPR.

Hållbarhetsaspekten beaktas genom att optimera bilder och undvika onödiga animationer, vilket minskar sidans energiförbrukning.

Övrigt

Webbsidan har testats på Chrome, Firefox och Safari.